

PRACTICAL 29 -

Objective - Objective - Operations on Binary Tree

1. **Create a Binary Tree**

Write a program to create a binary tree and perform level-wise insertion.

2. **Inorder Traversal (Recursive and Iterative)**

Traverse the binary tree in **inorder** fashion and display the result.

3. **Preorder Traversal (Recursive and Iterative)**

Traverse the binary tree in **preorder** and print the nodes.

4. **Postorder Traversal (Recursive and Iterative)**

Perform **postorder** traversal of the binary tree.

5. **Level Order Traversal (Breadth-First Search)**

Print the tree level by level using a queue.

6. **Count Leaf Nodes**

Write a function to count the number of leaf nodes in a binary tree.

7. **Find Height of Tree**

Calculate the maximum height (or depth) of a binary tree.

8. **Check if Binary Tree is Balanced**

Write a function to determine whether a binary tree is height-balanced. ### CODE -

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdbool.h>
4
5  // Definition of a node in the binary tree
6  typedef struct TreeNode {
7      int data;
8      struct TreeNode* left;
9      struct TreeNode* right;
10 } TreeNode;
11
12 // Create a new node
13 TreeNode* create_node(int data) {
14     TreeNode* new_node = (TreeNode*)malloc(sizeof(TreeNode));
15     new_node->data = data;
16     new_node->left = NULL;
17     new_node->right = NULL;
18     return new_node;
19 }
20
```

```
21 // Level-wise insertion in binary tree
22 TreeNode* insert_level_order(TreeNode* root, int data) {
23     TreeNode* new_node = create_node(data);
24     if (!root) return new_node;
25
26     TreeNode* temp;
27     TreeNode* queue[100];
28     int front = 0, rear = 0;
29     queue[rear++] = root;
30
31     while (front < rear) {
32         temp = queue[front++];
33         if (!temp->left) {
34             temp->left = new_node;
35             return root;
36         } else {
37             queue[rear++] = temp->left;
38         }
39         if (!temp->right) {
40             temp->right = new_node;
41             return root;
42         } else {
43             queue[rear++] = temp->right;
44         }
45     }
46     return root;
47 }
48
49 // Inorder Traversal (Recursive)
50 void inorder_recursive(TreeNode* root) {
51     if (root) {
52         inorder_recursive(root->left);
53         printf("%d ", root->data);
54         inorder_recursive(root->right);
55     }
56 }
57
58 // Inorder Traversal (Iterative)
59 void inorder_iterative(TreeNode* root) {
60     TreeNode* stack[100];
61     int top = -1;
62     TreeNode* current = root;
63
64     while (current || top != -1) {
65         while (current) {
66             stack[++top] = current;
67             current = current->left;
68         }
69         current = stack[top--];
70         printf("%d ", current->data);
71         current = current->right;
```

```
72     }
73 }
74
75 // Preorder Traversal (Recursive)
76 void preorder_recursive(TreeNode* root) {
77     if (root) {
78         printf("%d ", root->data);
79         preorder_recursive(root->left);
80         preorder_recursive(root->right);
81     }
82 }
83
84 // Preorder Traversal (Iterative)
85 void preorder_iterative(TreeNode* root) {
86     if (!root) return;
87
88     TreeNode* stack[100];
89     int top = -1;
90     stack[++top] = root;
91
92     while (top != -1) {
93         TreeNode* current = stack[top--];
94         printf("%d ", current->data);
95
96         if (current->right) stack[++top] = current->right;
97         if (current->left) stack[++top] = current->left;
98     }
99 }
100
101 // Postorder Traversal (Recursive)
102 void postorder_recursive(TreeNode* root) {
103     if (root) {
104         postorder_recursive(root->left);
105         postorder_recursive(root->right);
106         printf("%d ", root->data);
107     }
108 }
109
110 // Postorder Traversal (Iterative)
111 void postorder_iterative(TreeNode* root) {
112     if (!root) return;
113
114     TreeNode* stack1[100], *stack2[100];
115     int top1 = -1, top2 = -1;
116     stack1[++top1] = root;
117
118     while (top1 != -1) {
119         TreeNode* current = stack1[top1--];
120         stack2[++top2] = current;
121
122         if (current->left) stack1[++top1] = current->left;
```

```
123         if (current->right) stack1[++top1] = current->right;
124     }
125
126     while (top2 != -1) {
127         printf("%d ", stack2[top2--]->data);
128     }
129 }
130
131 // Level Order Traversal (Breadth-First Search)
132 void level_order_traversal(TreeNode* root) {
133     if (!root) return;
134
135     TreeNode* queue[100];
136     int front = 0, rear = 0;
137     queue[rear++] = root;
138
139     while (front < rear) {
140         TreeNode* current = queue[front++];
141         printf("%d ", current->data);
142
143         if (current->left) queue[rear++] = current->left;
144         if (current->right) queue[rear++] = current->right;
145     }
146 }
147
148 // Count Leaf Nodes
149 int count_leaf_nodes(TreeNode* root) {
150     if (!root) return 0;
151     if (!root->left && !root->right) return 1;
152
153     return count_leaf_nodes(root->left) + count_leaf_nodes(root->right)
154         ;
155 }
156
157 // Find Height of Tree
158 int find_height(TreeNode* root) {
159     if (!root) return 0;
160
161     int left_height = find_height(root->left);
162     int right_height = find_height(root->right);
163
164     return 1 + (left_height > right_height ? left_height : right_height
165         );
166 }
167
168 // Check if Binary Tree is Balanced
169 bool is_balanced(TreeNode* root) {
170     if (!root) return true;
171
172     int left_height = find_height(root->left);
173     int right_height = find_height(root->right);
```

```
172
173     if (abs(left_height - right_height) > 1) return false;
174
175     return is_balanced(root->left) && is_balanced(root->right);
176 }
177
178 // Main function to demonstrate all operations
179 int main() {
180     TreeNode* root = NULL;
181
182     // Create binary tree
183     root = insert_level_order(root, 1);
184     root = insert_level_order(root, 2);
185     root = insert_level_order(root, 3);
186     root = insert_level_order(root, 4);
187     root = insert_level_order(root, 5);
188     root = insert_level_order(root, 6);
189     root = insert_level_order(root, 7);
190
191     // Traversals
192     printf("Inorder Traversal (Recursive): ");
193     inorder_recursive(root);
194     printf("\nInorder Traversal (Iterative): ");
195     inorder_iterative(root);
196
197     printf("\nPreorder Traversal (Recursive): ");
198     preorder_recursive(root);
199     printf("\nPreorder Traversal (Iterative): ");
200     preorder_iterative(root);
201
202     printf("\nPostorder Traversal (Recursive): ");
203     postorder_recursive(root);
204     printf("\nPostorder Traversal (Iterative): ");
205     postorder_iterative(root);
206
207     printf("\nLevel Order Traversal: ");
208     level_order_traversal(root);
209
210     // Count leaf nodes
211     printf("\nNumber of Leaf Nodes: %d\n", count_leaf_nodes(root));
212
213     // Find height of tree
214     printf("Height of Tree: %d\n", find_height(root));
215
216     // Check if tree is balanced
217     printf("Is Tree Balanced: %s\n", is_balanced(root) ? "Yes" : "No");
218
219     return 0;
220 }
```

OUTPUT -

```
Inorder Traversal (Recursive): 4 2 5 1 6 3 7
Inorder Traversal (Iterative): 4 2 5 1 6 3 7
Preorder Traversal (Recursive): 1 2 4 5 3 6 7
Preorder Traversal (Iterative): 1 2 4 5 3 6 7
Postorder Traversal (Recursive): 4 5 2 6 7 3 1
Postorder Traversal (Iterative): 4 5 2 6 7 3 1
Level Order Traversal: 1 2 3 4 5 6 7
Number of Leaf Nodes: 4
Height of Tree: 3
```